École polytechnique fédérale de Lausanne

Semester project - spring 2026

Master in Applied Mathematics

Column subset selection methods in NLA with application to neural networks pruning

Author:
Yuekai Yan

Supervisor:
Laura Grigori
Zhipeng Xue

Contents

1 Introduction

The rapid growth of large-scale neural networks has motivated the use of numerical linear algebra (NLA) techniques for model compression, acceleration, and structured pruning [1]. Among these approaches, low-rank approximation methods are particularly attractive due to their ability to reduce model complexity while preserving important representation structures. However, for the model pruning task, many conventional low-rank approximation techniques, such as SVD-based and PCA-based methods [6], rely on factorized representations that may alter the original feature structure of the activation matrices. Existing pruning methods are also often heuristic and coarse-grained, lacking explicit approximation guarantees for the activation matrices.

In this context, methods for the Column Subset Selection Problem (CSSP) are especially appealing for neural network pruning. The advantages of CSSP-based pruning are twofold. First, column subset selection is inherently a low-rank approximation technique, enabling effective approximation of the activation matrices throughout the pruning procedure. This property helps preserve the activation representations and prediction behavior of the original model. Second, CSSP preserves the original column structure of the activation matrices. As a result, CSSP-based pruning can be naturally combined with other model compression techniques such as knowledge distillation [9] and quantization [10].

Motivated by these observations and the work [1], this project investigates the effectiveness and robustness of different column subset selection methods within the neural network pruning framework proposed in [1]. In particular, we study strong rank-revealing QR [7], RPCholesky [2], and adaptive randomized pivoting [3]. Our primary goal is to understand how different CSSP algorithms influence the pruning architecture, approximation quality, and numerical stability, and to compare the pruning performance of CSSP-based methods with classical pruning approaches such as ℓ_1/ℓ_2 filter pruning [12] and magnitude pruning [8].

The main contributions of this project are summarized as follows:

- Based on the model pruning framework of [1], we investigate the use of classical column subset selection methods as structured low-rank approximation techniques for neural network pruning.
- We compare several CSSP algorithms in terms of activation approximation quality and pruning performance.
- Through experiments with VGG-16 [13] on CIFAR-10 [11], we show that CSSP-based pruning consistently outperforms filter pruning under FLOPs constraints and magnitude pruning under parameter constraints, while further identifying discernible trends in the rank degradation exhibited by both approaches during pruning, revealing how spectral distortions propagate across layers and how different network blocks exhibit distinct sensitivities to compression.

The organization of this project is as follows. In Section 2, we introduce the notation and review the column subset selection problem. Section 3 presents several classical CSSP algorithms. In Section 4, we introduce our CSSP-based iterative pruning framework. Experimental results are presented in Section 5. Additional experimental figures are provided in Section A, and the usage of large language models in this project is described in Section B.

All experiments were performed in Python 3.12.8 on a workstation with an Apple M4 Pro CPU (14 cores), 48 GB of unified memory, and a 512 GB solid-state drive. The source code in this project is publicly available at `https://github.com/yuekai-yan/CSSP_Neural_Networks_Pruning`.

2 Preliminaries

In this section we briefly go through the basic knowledge of column subset selection problem (CSSP) and neural network pruning which are the basic concepts that will be referenced constantly in the paper.

2.1 Notations

We introduce here some general notations that we use throughout this project. Matrices are denoted by uppercase letters, while vectors are denoted by bold lowercase letters. We use MATLAB style notation for vectors and matrices. For example, consider a matrix A, for the i th to j th columns of A, we write $A(:, i : j)$. The Euclidean 2-norm of a vector $\mathbf{x}$ is denoted by $\|\mathbf{x}\|$, and the Frobenius norm of a matrix A is denoted by $\|A\|_F$. The transpose of a matrix A is written as $A^\top$. We denote by $\mathbf{0}_n$ the zero vector of length n, by $\mathbf{0}_{n\times m}$ the zero matrix of size $n \times m$, and by I_n the $n \times n$ identity matrix, the subscripts may be omitted if there is no ambiguity on the dimensions. The columns of the identity matrix of dimension n are denoted by $\mathbf{e}_1, \ldots, \mathbf{e}_n$. The singular values of A, arranged in nonincreasing order, are denoted by $\sigma_1(A), \ldots, \sigma_n(A)$. The range (or column space) of A is denoted by $\mathcal{R}(A)$.

For a neural network, given an input batch X, we denote by $Z_l = h_{1:l}(X; W^{(1)}, \ldots, W^{(l)})$ the activation matrix/tensor associated with the l-th learnable layer (e.g., linear and convolutional layers), evaluated after all subsequent parameter-free operations (e.g., ReLU, batch normalization, and pooling) attached to that layer.

2.2 Column Subset Selection Problem (CSSP)

The column subset selection problem (CSSP) is a classical problem in numerical linear algebra concerned with selecting a subset of columns from a matrix so that the selected columns provide the best possible approximation to the column space of the original matrix. More precisely, given a matrix $A \in \mathbb{R}^{m\times n}$, the objective of CSSP is to choose $r \ll \min\{m, n\}$ column indices $J = (j_1, \ldots, j_r)$ such that the subspace spanned by the corresponding columns $A(:, J)$ approximates the column space of A as accurately as possible, typically by minimizing

$$\|A - \Pi_J A\|_F, \tag{1}$$

where Π_J denotes the orthogonal projector onto $\mathcal{R}(A(:, J))$. Equivalently, this projector can be written as $A(:, J)A(:, J)^\dagger$.

Since $\mathcal{R}(\Pi_J A)$ has dimension at most r, the approximation error (1) cannot be smaller than the best rank-r approximation error:

$$\|A - AV_{\text{opt}}V_{\text{opt}}^\top\|_F^2 = \left(\sigma_{r+1}^2(A) + \cdots + \sigma_n^2(A)\right) \le \|A - \Pi_J A\|_F^2, \tag{2}$$

where matrix $V_{\text{opt}} \in \mathbb{R}^{n\times r}$ represents any orthonormal basis containing right singular vectors belonging to r largest singular values of A.

2.3 Neural Network Pruning

Neural network pruning aims to reduce the size and computational cost of a model by removing less important parameters while preserving its predictive performance. Existing pruning methods can generally be divided into two categories: structured pruning and unstructured pruning. Structured pruning removes entire neurons, channels, or other architectural units, resulting in a smaller and more efficient network. In contrast, unstructured pruning removes individual weights while keeping the network architecture unchanged. The following subsections briefly introduce these two paradigms and the representative methods considered in this project.

2.3.1 Structured Pruning

Since CSSP-based methods select and remove neurons or channels as whole units, they naturally belong to this category.

One of the most common structured pruning approaches is ℓ_1/ℓ_2 **filter pruning** [12]. It is a norm-based structured pruning method that ranks neurons or channels according to their ℓ_1 or ℓ_2 norms and removes those with the smallest values. After that, the corresponding neurons or input channels in the subsequent layer are removed accordingly to preserve the network architecture.

By explicitly removing neurons or channels from the network architecture, structured pruning reduces model redundancy while maintaining the regular structure of the network. This allows the compressed model to achieve lower memory usage and computational cost, making it an effective approach for model compression.

2.3.2 Unstructured Pruning

Unlike structured pruning, unstructured pruning preserves the original network architecture and introduces sparsity by setting selected individual weights to zero. Compared with structured pruning, unstructured pruning can often achieve higher compression rates, thus it is typically more parameter efficient. However, because the network architecture and tensor dimensions remain unchanged, the model is not physically compressed in the same way as structured pruning, and the resulting sparse models usually require specialized hardware or software support to fully realize computational speedups.

One of the most widely used unstructured pruning approaches is **magnitude pruning** [8]. This method ranks individual weights according to their absolute values and removes those with the smallest magnitudes. The underlying assumption is that weights with small magnitudes contribute less to the network output and can therefore be removed with limited impact on model performance.

3 Methods for Column Subset Selection Problems

3.1 Pivoted QR decomposition

QR factorization with column pivoting (QRCP) [6] is one of the most widely used deterministic methods for the column subset selection problem (CSSP). Given a matrix $A \in \mathbb{R}^{m\times n}$, QRCP computes a factorization

$$AP = QR,$$

where P is a permutation matrix that reorders the columns of A according to their importance. At each step, QRCP greedily selects the column with the largest residual ℓ_2-norm and uses it as the next pivot. The selected pivot columns define a low-dimensional subspace that captures the dominant structure of the matrix.

Due to its simplicity, numerical stability, and effectiveness in identifying representative columns, QRCP is one of the most widely used methods for CSSP, with an arithmetic complexity of $O(mnr)$ for computing a rank-r approximation.

3.2 Strong Rank Revealing QR

Strong rank revealing QR (StrongRRQR) [7] can be viewed as a stronger version of QRCP, with additional guarantees on the quality of the selected columns and the conditioning of the resulting factors. StrongRRQR aims to compute, for a matrix $A \in \mathbb{R}^{m\times n}$, a decomposition of the form:

$$AP = Q\begin{pmatrix} A_r & B_r \\ & C_r \end{pmatrix},$$

where $Q \in \mathbb{R}^{m\times m}$ is orthogonal, $A_r \in \mathbb{R}^{r\times r}$ is upper triangular with nonnegative diagonal elements, $B_r \in \mathbb{R}^{r\times(n-r)}$, $C_r \in \mathbb{R}^{(m-r)\times(n-r)}$, and P is a permutation matrix chosen to reveal linear dependence among the columns of A, and the factorization satisfies

$$\sigma_i(A_r) \geq \frac{\sigma_i(A)}{\sqrt{1+f^2r(n-r)}}, \quad \sigma_j(C_r) \leq \sigma_{r+j}(A)\sqrt{1+f^2r(n-r)}, \quad \left|(A_r^{-1}B_r)_{i,j}\right| \leq f, \quad f \geq 1. \tag{3}$$

For a nonsingular matrix A, $1/\omega_i(A)$ denotes the 2-norm of the i-th row of A^{-1}, and $\gamma_j(A)$ denotes the 2-norm of the j-th column of A. Let

$$\rho(R,r) = \max_{1\leq i\leq r,\ 1\leq j\leq n-r} \sqrt{(A_r^{-1}B_r)_{i,j}^2 + \left(\frac{\gamma_j(C_r)}{\omega_i(A_r)}\right)^2},$$

the StrongRRQR algorithm is summarized in Algorithm 1 and requires $O(mn\min(m,n) + sm\min(m,n))$ flops, where s denotes the number of column interchanges performed in the `while` loop.

Theorem 3.2 in [7] proves Algorithm 1 satisfies Equation (3), let J denotes the column indices selected by StrongRRQR, $Q = [Q_1, Q_2]$, thus $A(:,J) = Q_1A_r$ and $\Pi_J = Q_1Q_1^\top$, it follows that

$$\begin{aligned}
\|A - \Pi_J A\|_F^2 &= \|AP - \Pi_J AP\|_F^2 \\
&= \|AP\|_F^2 - \|\Pi_J AP\|_F^2 \\
&= \|A_r\|_F^2 + \|B_r\|_F^2 + \|C_r\|_F^2 - \|A_r\|_F^2 - \|B_r\|_F^2 \\
&= \|C_r\|_F^2 \\
&\leq \sum_{j=1}\sigma_j^2(C_r) = \left(1+f^2r(n-r)\right)\sum_{j>r}\sigma_j^2(A).
\end{aligned}$$

3.3 Randomly Pivoted Cholesky

Randomly Pivoted Cholesky (RPCholesky) [2] is a randomized low-rank approximation algorithm for positive semidefinite (psd) matrices. Given a psd matrix $G \in \mathbb{R}^{n\times n}$, the goal is to construct a rank-r approximation $G \approx FF^*$ (Nyström approximation). The motivation for this approach is that, in many large-scale applications, directly factorizing G is computationally expensive, whereas diagonal entries and individual columns can often be accessed much more efficiently.

Compared with deterministic pivoted Cholesky, RPCholesky replaces greedy pivot selection with randomized sampling, thereby reducing the cost of pivot search while still favoring informative columns. At each

Algorithm 1: StrongRRQR (Algorithm 4 in [7])

Input: Matrix $A \in \mathbb{R}^{m\times n}$, approximation rank r, bounding constant f
Output: Matrix $R \in \mathbb{R}^{r\times n}$, permutation matrix $P \in \mathbb{R}^{n\times n}$
Compute

$$R \equiv \begin{pmatrix} A_r & B_r \\ & C_r \end{pmatrix} := \mathcal{R}_r(A), \quad P = I$$

while $\rho(R, r) > f$ **do**
  Find indices i, j such that

$$\sqrt{\left((A_r^{-1}B_r)_{i,j}\right)^2 + \left(\frac{\gamma_j(C_r)}{\omega_i(A_r)}\right)^2} > f$$

  Compute

$$R \equiv \begin{pmatrix} A_r & B_r \\ & C_r \end{pmatrix} := \mathcal{R}_r(R\Pi_{i,j+r})$$

  Update permutation: $P \leftarrow P\,P_{i,j+r}$
end
return R, P

step, RPCholesky samples a pivot index with probability proportional to the current residual diagonal, which reflects the amount of information not yet captured by the current approximation. It then evaluates the corresponding column of G, removes the overlap with previously selected components, and normalizes the resulting residual column to update the factor F. Repeating this procedure produces a low-rank approximation that incrementally captures the dominant structure of the matrix, as shown in Algorithm 2. The arithmetic operation of RPCholesky is $O(r^2 n)$. RPCholesky gives the following trace norm error bound

Theorem 3.1 (Theorem 5.1 in [2])**.** *Let G be a positive semidefinite (psd) matrix. Fix $c \in \mathbb{N}$ and $\varepsilon > 0$. The rank-r column Nyström approximation $\hat{G}$ produced by r steps of RPCholesky attains the bound*

$$\mathbb{E}\ \mathrm{tr}\left(G - \hat{G}\right) \le (1+\varepsilon)\sum_{i>c}\lambda_i(G), \tag{4}$$

provided that the number k of columns satisfies

$$r \ge \frac{c}{\varepsilon} + \min\left\{c\log\left(\frac{1}{\varepsilon\eta}\right),\ c + c\log_+\left(\frac{2^c}{\varepsilon}\right)\right\}. \tag{5}$$

The relative error η is defined by $\eta := \frac{\mathrm{tr}(G-[\![G]\!]_c)}{\mathrm{tr}(G)}$. $\log_+(x) := \max\{\log x, 0\}, x > 0$, and the logarithm has base e.

Algorithm 2: RPCholesky (Algorithm 1 in [2]))

Input: SPD matrix $G = A^\top A \in \mathbb{R}^{n\times n}$, approximation rank r
Output: Pivot set $J = (j_1, \ldots, j_r)$, matrix $F \in \mathbb{R}^{n\times r}$ defining the Nyström approximation $\hat{G} = FF^\top$
Initialize $F \leftarrow 0_{n\times r}$, $d \leftarrow \mathrm{diag}(G)$, and $J \leftarrow ()$;
for $k = 1 : r$ **do**
  Sample pivot $j_k \sim d/\sum_{j=1}^N d(j)$;
  Append j_k to J ;
  $g \leftarrow G(:, j_k)$;
  $g \leftarrow g - F(:, 1:k-1)\,F(j_k, 1:k-1)^*$;
  $F(:,k) \leftarrow g/\sqrt{g(j_k)}$;
  $d \leftarrow d - |F(:,k)|^2$;
  $d \leftarrow \max(d, 0)$;
end
return J, F

In essence, the above algorithm computes the column Nyström approximation of G:

$$\begin{aligned}\hat{G} = FF^\top &= G(:,J)G(J,J)^\dagger G(J,:) \\ &= G(:,J)(I(J,:)GI(:,J))^\dagger G(J,:) \\ &= A^\top A(:,J)A(:,J)^\dagger (A(:,J)^\top)^\dagger A(:,J)^\top A \\ &= (\Pi_J A)^\top \Pi_J A,\end{aligned}$$

thus computing the column Nyström approximation of $G = A^\top A$ is equivalent to computing the projection of A onto the subset J, which is known as **Gram correspondence**. This equivalence also connects the trace-norm error of the Nyström approximation to the CSSP projection error. Indeed,

$$\begin{aligned}\mathrm{tr}(G - \hat{G}) &= \mathrm{tr}(G) - \mathrm{tr}(\hat{G}) \\ &= \|A\|_F^2 - \|F\|_F^2 \\ &= \|A\|_F^2 - \|\Pi_J A\|_F^2 \\ &= \|A - \Pi_J A\|_F^2 \,.\end{aligned}$$

Therefore, minimizing the trace-norm error of G is equivalent to minimizing the projection error in CSSP, which justifies the use of RPCholesky for column subset selection. Moreover, by the Gram correspondence, Algorithm 2 admits an equivalent formulation directly on the matrix A, namely Randomly Pivoted QR (RPQR) (Algorithm 3 [5]). In the context of CSSP for a matrix $A \in \mathbb{R}^{m\times n}$, one may either construct the Gram matrix $G = A^\top A$ and apply RPCholesky to G, or equivalently apply RPQR directly to A, thereby reducing the total arithmetic complexity from $O(n^2m + r^2n)$ to $O(mnr)$.

Algorithm 3: Randomly Pivoted QR [5]

Input: Matrix $A \in \mathbb{R}^{m\times n}$, approximation rank r
Output: Pivot set $J = (j_1, \ldots, j_r)$, projection approximation $\hat{A} = \Pi_J A \in \mathbb{R}^{m\times n}$
Initialize $\hat{A} \leftarrow 0_{m\times n}$, $R \leftarrow A$ and $J \leftarrow ()$;
for $k = 1 : r$ **do**
  Sample pivot $j_k \sim \|R(:,j_k)\|^2/\|R\|_F^2$;
  Append j_k to J ;
  $\hat{A} \leftarrow \hat{A} + R(:,j_k)R(:,j_k)^\top R/\|R(:,j_k)\|^2$;
  $R \leftarrow R - R(:,j_k)R(:,j_k)^\top R/\|R(:,j_k)\|^2$;
end
return $J, \hat{A}$

3.4 Adaptive Randomized Pivoting

Adaptive randomized pivoting (ARP) [3] is a randomized strategy for CSSP. Instead of performing pivot selection directly on the original matrix $A \in \mathbb{R}^{m\times n}$, ARP first constructs an orthonormal basis $V \in \mathbb{R}^{n\times r}$ that approximates the dominant row space of A, for example by sketching $A^\top$ with a random matrix Θ, computing a QR factorization $A^\top\Theta = QR$, and setting $V = Q$. More precisely, V is chosen such that

$$\|A - AVV^\top\|_F \approx 0.$$

The main motivation of ARP is that operating on the compressed basis V is significantly cheaper than working directly with the full matrix A, while still preserving the dominant spectral structure of A. ARP then performs Algorithm 3 on V: at each iteration, a pivot index is sampled with probability proportional to the squared row norms of the current residual basis, followed by an orthogonal projection update that removes the selected direction. Furthermore, since ARP operates on a matrix with orthonormal columns, the denominator in the sampling step becomes an exact quantity, which makes the whole process much more stable, and Algorithm 3 can be rewritten as Algorithm 4. When applied to $V \in \mathbb{R}^{n\times r}$, the arithmetic cost of ARP is $O(r^2n)$.

The low rank approximation provided by ARP satisfies

Theorem 3.2 (Theorem 2.4 in [3])**.** *Let $A \in \mathbb{R}^{m\times n}$ and let $V \in \mathbb{R}^{n\times r}$ be an orthonormal basis. Then the random index set J returned by Algorithm 4 satisfies*

$$\mathbb{E}\left[\|A - \Pi_J A\|_F^2\right] \le \mathbb{E}\left[\left\|A - A(:,J)V(J,:)^{-T}V^T\right\|_F^2\right] = (r+1)\left\|A - AVV^T\right\|_F^2 . \tag{6}$$

Algorithm 4: Adaptive Randomized Pivoting (Algorithm 2.1 in [3])

Input: Matrix $V \in \mathbb{R}^{n\times r}$ with orthonormal columns
Output: Indices $J = (j_1, \ldots, j_r)$
Initialize $V_0 = V$ and $J_0 = ()$;
for $k = 1 : r$ **do**
  Set $p_j = \|V_{k-1}(j,:)\|_2^2/(r-k+1)$ for $j = 1, \ldots, n$;
  Sample index j_k according to probabilities p_j;
  Set $J_k \leftarrow (J_{k-1}, j_k)$;
  Update $V_k \leftarrow V_{k-1}\Big(I - V_{k-1}(j_k,:)^\dagger V_{k-1}(j_k,:)\Big)$;
end
Set $J \leftarrow J_r$;
return J

3.5 Volume Sampling

Volume sampling [5] is a probability distribution over k-subsets of rows of a matrix, where each subset is selected with probability proportional to the squared volume spanned by the chosen rows. Equivalently, these probabilities can be expressed in terms of determinants of the corresponding Gram matrices, see Definition 3.1.

Originally introduced in the context of low-rank matrix approximation, volume sampling favors diverse and linearly independent subsets, since sets of nearly dependent rows generate small volume and therefore receive low probability. As a result, it provides a natural and effective mechanism for selecting representative rows or columns that preserve the essential geometric and algebraic structure of the original matrix.

Definition 3.1 (Volume sampling)**.** Fix a matrix $A \in \mathbb{R}^{m\times n}$ and an integer $r \le \min(m,n)$. A random subset $J \subseteq \{1, \ldots, m\}$ with $|J| = r$ is said to follow the volume sampling distribution, denoted by $J \sim \mathrm{VS}_r(A)$, if

$$\mathbb{P}\{J = S\} = \frac{\mathrm{Vol}(S)^2}{\sum_{|R|=r}\mathrm{Vol}(R)^2}, \qquad \text{where } \mathrm{Vol}(T) := |\det A(T,:)|.$$

Equivalently, this distribution can be expressed as

$$\mathbb{P}\{J = S\} = \frac{\det H(S,S)}{\sum_{|R|=r}\det H(R,R)}, \qquad \text{where } H = AA^\top \text{is the Gram matrix of A.}$$

Deduced from Definition 3.1, when regarding volume sampling as a distribution over ordered sequences of length r, namely $j_1, j_2, \ldots, j_r$ instead of unordered r-subsets, we have the following probability:

$$\mathbb{P}\{J = (j_1, j_2, \ldots, j_r)\} = \begin{cases} \dfrac{\det(A(J,:)A(J,:)^\top)}{r!\displaystyle\sum_{|S|=r}\det\Big(A(S,:)A(S,:)^\top\Big)}, & \text{if } j_1, \ldots, j_r \text{ are distinct,} \\ 0, & \text{otherwise.} \end{cases} \tag{7}$$

Then we have the following lemma for the marginal probabilities:

Lemma 3.1 (Lemma 14 in [4])**.** *Assume that* $\mathbb{P}\{S = (j_1, \ldots, j_{k-1})\} > 0$*, for a random r-tuple* $J = (j_1, j_2, \ldots, j_r)$ *from the volume sampling over r-tuples. Let* $B = A - A\Pi_S, C_{j_k} = B - B\Pi_{\{j_k\}} = A - A\Pi_{S\cup\{j_k\}}$*. Then,*

$$\mathbb{P}\{j_k \mid j_1, \ldots, j_{k-1}\} = \frac{\|B(j_k,:)\|^2\left|c_{m-r+k}(C_{j_k}C_{j_k}^T)\right|}{(r-k+1)\left|c_{m-r+k-1}(BB^T)\right|}, \tag{8}$$

where for any matrix $M \in \mathbb{R}^{m\times n}$*,* $c_{m-k}(MM^\top)$ *denotes the coefficient of* x^{m-k} *in the characteristic polynomial*

$$\det(xI - MM^\top) = x^m + c_{m-1}(MM^\top)x^{m-1} + \cdots + c_0(MM^\top).$$

Based on Lemma 3.1, we have the following volume sampling algorithm:

Algorithm 5: Volume Sampling (Algorithm 1 in [4])

Input: A matrix $A \in \mathbb{R}^{m\times n}$ and an integer $1 \le r \le \operatorname{rank}(A)$
Output: A subset J of r rows of A, sampled with probability proportional to $\det(A(J,:)A(J,:)^\top)$
Initialize $B = A$ and $J_0 = ()$;
for $k = 1 : r$ **do**
  Set $p_j = \|B(j,:)\|^2 \cdot \left|c_{m-r+k}(C_j C_j^T)\right|$ for $j = 1, \dots, m$, where $C_j = B - B\Pi_{\{j\}}$ is obtained by projecting each row of B orthogonally to $B(j,:)$;
  Sample index j_k with probability proportional to p_j;
  Set $J_k \leftarrow (J_{k-1}, j_k)$;
  Update $B \leftarrow C_{j_k}$;
end
Set $J \leftarrow J_r$;
return J

3.6 Equivalence of Adaptive Randomized Pivoting and Volume Sampling

In this section, we establish the equivalence between Algorithm 4 and Algorithm 5 by showing that they generate the same sampling distribution when applied to the same matrix V.

In particular, when the input to Algorithm 5 is the truncated singular value decomposition of $A \in \mathbb{R}^{m\times n}$, given by

$$A_r = U_r \Sigma_r V_r^\top,$$

where $\Sigma_r \in \mathbb{R}^{r\times r}$ is the diagonal matrix with the diagonal entries $\sigma_1(A) > \sigma_2(A) > \cdots > \sigma_r(A) > 0$, $U_r \in \mathbb{R}^{m\times r}$ and $V_r \in \mathbb{R}^{n\times r}$ are the matrices formed by the leading r left singular vectors and right singular vectors of A, respectively. Since for any r-tuple S, $V_r(S,:)$ is an $r \times r$ matrix,

$$\begin{aligned}
\det\Big(A_r(:,S)^\top A_r(:,S)\Big) &= \det\Big(V_r(S,:)\Sigma_r U_r^\top U_r \Sigma_r V_r(S,:)^\top\Big) \\
&= \det\Big(V_r(S,:)\Sigma_r^2 V_r(S,:)^\top\Big) \\
&= \det\Big(\Sigma_r^2\Big)\det\Big(V_r(S,:)V_r(S,:)^\top\Big),
\end{aligned}$$

by Equation (7), the sampling distribution induced by volume sampling on $A_r^\top$ coincides with that induced by ARP on V_r. Hence, the two algorithms are equivalent in this setting.

Furthermore, if the input to Algorithm 4 is obtained via sketching $A^\top$, i.e., $A^\top\Theta = VR$ is a QR decomposition, where $\Theta \in \mathbb{R}^{m\times r}$ is a randomized subspace embedding, then this equivalence can be interpreted as Algorithm 5 operating on the sketched matrix $A^\top\Theta$.

To establish this equivalence, it suffices to show that ARP induces exactly the volume sampling distribution when both algorithms are applied to the same matrix $V \in \mathbb{R}^{n\times r}$. The proof is given below.

Proof of distributional equivalence In the k-th loop of Algorithm 4, we have

$$\mathbb{P}\{j_k \mid j_1, \dots, j_{k-1}\} = \frac{\|V_{k-1}(j_k,:)\|^2}{r-k+1}. \tag{9}$$

Let $Q = VV^\top$ and $X = V(J_{k-1},:)^\top$, since ARP selects linearly independent rows at each step, X has full column rank $k-1$, we can write the orthogonal projector onto $\mathcal{R}(X)$ as:

$$XX^\dagger = X(X^\top X)^{-1}X^\top = XQ(J_{k-1}, J_{k-1})^{-1}X^\top.$$

Thus by Pythagoras,

$$\begin{aligned}
\|V_{k-1}(j_k,:)\|^2 &= \|V(j_k,:)\|^2 - \|V(j_k,:)XX^\dagger\|^2 \\
&= Q(j_k,j_k) - V(j_k,:)XX^\dagger(XX^\dagger)^\top V(j_k,:)^\top \\
&= Q(j_k,j_k) - V(j_k,:)XX^\dagger V(j_k,:)^\top \\
&= Q(j_k,j_k) - V(j_k,:)XQ(J_{k-1},J_{k-1})^{-1}X^\top V(j_k,:)^\top,
\end{aligned}$$

where $Q(j_k,j_k) - V(j_k,:)XQ(J_{k-1},J_{k-1})^{-1}X^\top V(j_k,:)^\top$ is the **Schur complement** of $Q(J_{k-1},J_{k-1})$ in the matrix

$$\begin{pmatrix} Q(J_{k-1},J_{k-1}) & X^\top V(j_k,:)^\top \\ V(j_k,:)X & Q(j_k,j_k) \end{pmatrix} = \begin{pmatrix} Q(J_{k-1},J_{k-1}) & Q(j_k,J_{k-1})^\top \\ Q(j_k,J_{k-1}) & Q(j_k,j_k) \end{pmatrix} = Q(J_k,J_k).$$

Thus we deduce that

$$\begin{aligned}\|V_{k-1}(j_k,:)\|^2 &= Q(j_k,j_k) - V(j_k,:)XQ(J_{k-1},J_{k-1})^{-1}X^\top V(j_k,:)^\top \\ &= \det\Big(Q(j_k,j_k) - V(j_k,:)XQ(J_{k-1},J_{k-1})^{-1}X^\top V(j_k,:)^\top\Big) \\ &= \frac{\det\left(Q(J_k,J_k)\right)}{\det\left(Q(J_{k-1},J_{k-1})\right)}.\end{aligned}$$

Combined with Equation (9), it follows that

$$\begin{aligned}\mathbb{P}\{J=(j_1,j_2,\ldots,j_r)\} &= \mathbb{P}\{j_1\}\mathbb{P}\{j_2 \mid j_1\}\cdots\mathbb{P}\{j_r \mid j_1,\ldots,j_{r-1}\} \\ &= \frac{\det\left(Q(J_r,J_r)\right)}{r!} \\ &= \frac{\det\left(V(J_r,:)V(J_r,:)^\top\right)}{r!}.\end{aligned} \tag{10}$$

Since V has orthonormal columns, $V^\top V = I_r$. By the Cauchy–Binet formula,

$$1 = \det(V^\top V) = \sum_{S\subseteq[n]:\,|S|=r} \det\Big(V(S,:)^\top\Big)\det(V(S,:)) = \sum_{S\subseteq[n]:\,|S|=r} \det\Big(V(S,:)V(S,:)^\top\Big),$$

which shows that (10) coincides with the probability formula (7) for volume sampling.

Table 1: Output activation tensor shapes and parameter tensor shapes for different layer types.

Layer Type	Output Activation Shape	Parameter Shape
Linear	(batch size, out neurons)	(out neurons, in neurons)
Conv	(batch size, out channels, out height, out width)	(out channels, in channels, kernel size, kernel size)

4 CSSP-Based Pruning Framework

In this section, we introduce a CSSP-based pruning framework adapted from [1] as a structured low-rank approximation technique for neural network pruning. Classically, the Singular Value Decomposition (SVD) (see, e.g., [6]) provides an optimal low-rank approximation. However, in our setting, pruning operates on activation matrices obtained after nonlinear transformations. As a result, the low-rank basis vectors produced by SVD generally correspond to linear combinations of neurons or channels rather than existing neurons or channels in the network, making SVD unsuitable for directly performing structured pruning. In contrast, column subset selection constructs a structured low-rank approximation of a matrix Z, where the approximation basis is a subset of the columns of Z.

Throughout this paper, we follow the PyTorch convention for tensor layouts. The output activation shapes after different layer types and the parameter tensor shapes for different layer types are summarized in Table 1.

4.1 CSSP-Based Iterative Pruning Algorithm

We adopt an iterative pruning strategy that adaptively selects the most suitable layer for pruning at each iteration. The overall objective is to preserve the activation structure of the original network as much as possible while reducing computational or parameter complexity.

At each pruning step, we compute a pruning score for every layer based on two criteria: the intrinsic low-rank structure of its activation matrix and the computational cost or parameter count of the layer. The layer with the smallest pruning score is then selected for pruning, corresponding to a layer that is both highly compressible and associated with a high computational or parameter cost. To preserve the original prediction space, the final output layer is excluded from the candidate set and is never selected for pruning.

For the selected layer, pruning is always performed on the activation matrix after the nonlinear transformation, i.e., after the ReLU operation (and after pooling for convolutional layers). Rather than directly removing the corresponding neurons or channels in subsequent layers, we propagate the pruning effect through a correction matrix. This correction mechanism transfers the approximation error induced by pruning to the next layer in a controlled manner, thereby mitigating performance degradation.

Assume l is the chosen layer index at one step and Z_l is the activation matrix of the first l layers of size (batch size, out neurons), i.e. $Z_l = h_{1:l}(X; W^{(1)}, \dots, W^{(l)})$ (if layer l is a convolution layer, we reshape Z_l of size (batch size $\times$ out height $\times$ out width, out channels)), using **column subset selection methods**, we select the layer index J of Z_l or Reshape(Z_l) to be kept, we have

$$Z_l(:, J) = \mathrm{ReLU}(Z_{l-1}(W^{(l)})^\top)(:, J) = \mathrm{ReLU}(Z_{l-1}(W^{(l)}(J, :))^\top) = \mathrm{ReLU}(Z_{l-1}(\hat{W}^{(l)})^\top)$$

we can also get the correction matrix T simultaneously for the next layer:

$$T = Z_l(:, J)^\dagger Z_l.$$

Since our methods guarantee the small $\|Z_l - Z_l(:, J)Z_l(:, J)^\dagger Z_l\|_F$, we can then update the next-layer weight matrix using $W^{(l+1)}T^\top$:

$$Z_l(W^{(l+1)})^\top \approx Z_l(:, J)Z_l(:, J)^\dagger Z_l(W^{(l+1)})^\top = Z_l(:, J)(W^{(l+1)}T^\top)^\top = Z_l(:, J)(\hat{W}^{(l+1)})^\top{}^1. \tag{11}$$

The whole process is summarized in Algorithm 6.

[1]For convolutional layers, $\hat{W}^{(l+1)}$ is updated by $W^{(l+1)} \times_{\text{in}} T^\top$, $\times_{\text{in}}$ denotes tensor contraction over the input-channel dimension: $\hat{W}^{(l+1)}_{o,k,h,w} = \sum_m W^{(l+1)}_{o,m,h,w} T_{k,m}$. In PyTorch, this operation can be implemented as `torch.einsum('km,omhw->okhw', T, W_next)`, where `W_next` denotes $W^{(l+1)}$.

Algorithm 6: Iterative Pruning

Input: Neural net $h(x; W^{(1)}, \ldots, W^{(L)})$, layers to not prune $S \subset [L]$, pruning set X, remaining fraction ρ, step size λ

Output: Pruned network $h(x; \hat{W}^{(1)}, \ldots, \hat{W}^{(L)})$

$F_0 \leftarrow$ Compute_Recourse$(h(x; W^{(1)}, \ldots, W^{(L)}))$;

for $l \in \{1, \ldots, L\}$ **do**

  $\hat{W}^{(l)} \leftarrow W^{(l)}$;

end

while $F > \rho F_0$ **do**

  $S, K \leftarrow \{\}$;

  for $l \in \{1, \ldots, L\} \setminus S$ **do**

    $Z_l \leftarrow h_{1:l}(X; \hat{W}^{(1)}, \ldots, \hat{W}^{(l)})$;

    $R \leftarrow$ Pivot_QR(Z_l) {reshape if conv layer} ;

    $k \leftarrow$ num_channel$(\hat{W}^{(l)}) \times \lambda$;

    $Err_l \leftarrow \left|\frac{R[k+1,k+1]}{R[1,1]}\right|$;

    $F_l \leftarrow$ Compute_Resourse$(\hat{W}^{(l)}, \hat{W}^{(l+1)})$;

    S.append(Err_l / F_l);

    K.append(k);

  end

  $l \leftarrow \arg\min(S)$;

  $(J, T) \leftarrow$ Column Subset Selection(Z_l) {reshape if conv layer} ;

  $\hat{W}^{(l)} \leftarrow \hat{W}^{(l)}(J, :)$ {$\hat{W}^{(l)}(J, :, :, :)$ if conv layer};

  if *next layer is not a Flatten layer* **then**

    $\hat{W}^{(l+1)} \leftarrow \hat{W}^{(l+1)} T^\top$ {$\hat{W}^{(l+1)} \times_{\text{in}} T^\top$ if conv layer} ;

  else

    $T \leftarrow T \otimes I$;

    $\hat{W}^{(l+1)} \leftarrow \hat{W}^{(l+1)} T^\top$;

  end

  $F \leftarrow$ Compute_Resourse$\left(h(x; \hat{W}^{(1)}, \ldots, \hat{W}^{(L)})\right)$;

end

4.2 Error Propagation Analysis of CSSP-Based Pruning

We now analyze the error propagated to the next layer's activation matrix. Since $\mathrm{ReLU}(x)$ is 1-Lipschitz, by Equation (11), it follows that

$$\begin{aligned} Z_{l+1} - \hat{Z}_{l+!} &= \|\mathrm{ReLU}(Z_l (W^{(l+1)})^\top) - \mathrm{ReLU}(Z_l(:, J) Z_l(:, J)^\dagger Z_l (W^{(l+1)})^\top)\|_F \\ &\le \|Z_l (W^{(l+1)})^\top - Z_l(:, J) Z_l(:, J)^\dagger Z_l (W^{(l+1)})^\top\|_F \\ &\le \|Z_l - Z_l(:, J) Z_l(:, J)^\dagger Z_l\|_F \; \|(W^{(l+1)})^\top\|_F. \end{aligned} \tag{12}$$

The error $\|Z_l - Z_l(:, J) Z_l(:, J)^\dagger Z_l\|_F$ depends on the specific column subset selection method used. Through the iterative pruning procedure, the low-rank approximation property of column subset selection ensures that the propagated error in subsequent layers remains controlled up to multiplication by the next-layer weight matrix, thereby keeping the resulting activation representations close to those of the original network.

Since, in many classification and probabilistic prediction tasks, the final prediction is determined by the row-wise argmax of the final activation matrix, preserving the activation representations with small Frobenius error makes CSSP-based pruning more likely to preserve the original classification decisions. Therefore, we expect CSSP-based pruning to achieve better predictive performance than conventional pruning methods, which are typically more coarse-grained and do not explicitly preserve low-rank activation structures.

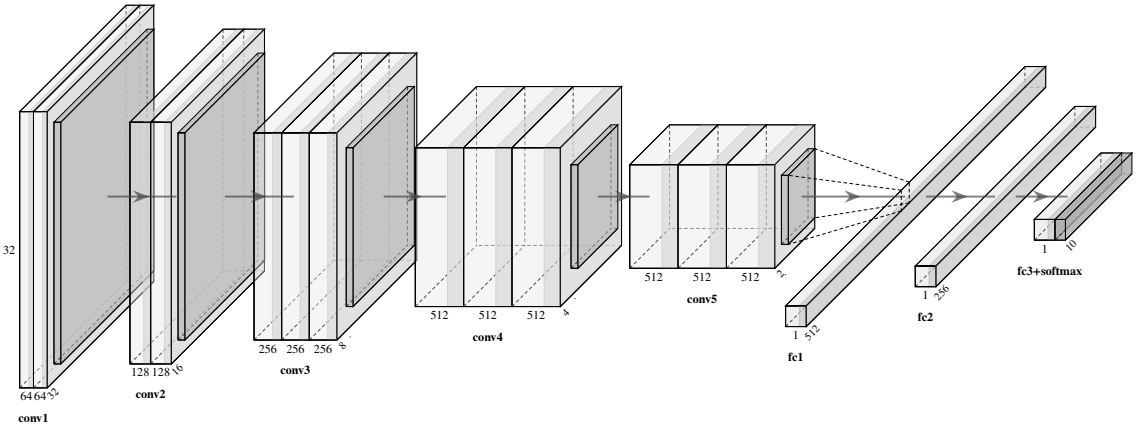

Figure 7: Architecture of VGG-16 used in the experiments.

5 Experiments

In the experiments, we use the VGG-16 [13] architecture (Figure 7) trained on CIFAR-10 [11] as the baseline model for a **10-class classification task**. The pretrained network achieves a classification accuracy of 91.13% with a loss of 0.4497 on test set and serves as the starting point for all subsequent pruning experiments. The layer-wise output sizes, parameter counts, and FLOPs complexity for VGG-16 are summarized in Table 2.

Starting from the pretrained baseline model, we apply Algorithm 6 under different pruning strategies and resource constraints. Besides the proposed CSSP-based methods, we consider two representative pruning techniques: ℓ_1/ℓ_2 filter pruning for structured pruning and magnitude pruning for unstructured pruning. The experiments are divided into two groups according to the optimization objective. Under FLOPs constraints, CSSP-based pruning is compared with ℓ_1/ℓ_2 filter pruning, while under parameter-count constraints it is compared with magnitude pruning.

5.1 CSSP-Based Pruning vs. Filter Pruning under FLOPs Constraints

We first compare three CSSP-based pruning methods (StrongRRQR, RPCholesky, and ARP) with ℓ_1/ℓ_2 filter pruning under FLOPs constraints. To ensure a fair comparison, filter pruning is implemented within the same iterative pruning framework as Algorithm 6. In both approaches, layer selection is based on a prunability score that balances layer compressibility against computational cost. The only difference is that CSSP-based methods measure compressibility through a rank-based criterion derived from the activation matrix, whereas filter pruning uses an ℓ_1/ℓ_2-norm-based criterion computed from the layer weights.

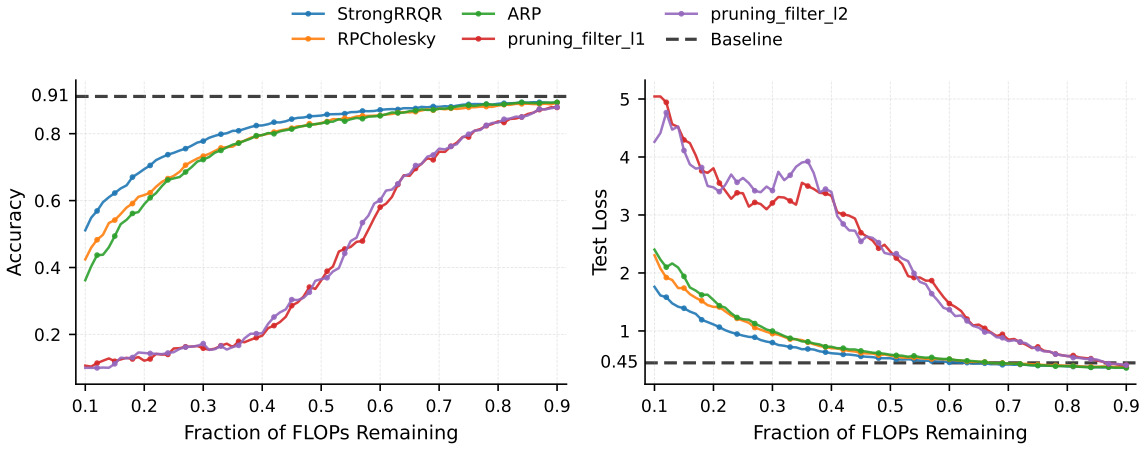

Figure 8: Performance comparison of different compression methods across varying FLOPs ratios. The left panel shows test accuracy, while the right panel shows test loss. The dashed line indicates the uncompressed baseline.

Table 2: Layer-wise output sizes, parameter counts, and FLOPs for VGG-16.

Layer	Output size	Params	FLOPs
ConvBNReLU 1-1	$64 \times 32 \times 32$	1,920	3.67M
ConvBNReLU 1-2	$64 \times 32 \times 32$	37,056	75.63M
MaxPool 1	$64 \times 16 \times 16$	0	0.07M
ConvBNReLU 2-1	$128 \times 16 \times 16$	74,112	37.81M
ConvBNReLU 2-2	$128 \times 16 \times 16$	147,840	75.56M
MaxPool 2	$128 \times 8 \times 8$	0	0.03M
ConvBNReLU 3-1	$256 \times 8 \times 8$	295,680	37.78M
ConvBNReLU 3-2	$256 \times 8 \times 8$	590,592	75.53M
ConvBNReLU 3-3	$256 \times 8 \times 8$	590,592	75.53M
MaxPool 3	$256 \times 4 \times 4$	0	0.02M
ConvBNReLU 4-1	$512 \times 4 \times 4$	1,181,184	37.76M
ConvBNReLU 4-2	$512 \times 4 \times 4$	2,360,832	75.51M
ConvBNReLU 4-3	$512 \times 4 \times 4$	2,360,832	75.51M
MaxPool 4	$512 \times 2 \times 2$	0	0.01M
ConvBNReLU 5-1	$512 \times 2 \times 2$	2,360,832	18.88M
ConvBNReLU 5-2	$512 \times 2 \times 2$	2,360,832	18.88M
ConvBNReLU 5-3	$512 \times 2 \times 2$	2,360,832	18.88M
MaxPool 5	$512 \times 1 \times 1$	0	0.002M
Flatten	512	0	0
LinearBNReLU 1	512	263,680	0.53M
LinearBNReLU 2	256	131,840	0.26M
Linear	10	2,570	0.01M
Total	–	15,121,226	627.86M

Figure 8 presents the test accuracy and test loss achieved by different pruning methods across varying FLOPs retention ratios. The results show that CSSP-based pruning methods consistently outperform ℓ_1/ℓ_2 filter pruning, achieving higher classification accuracy and lower test loss throughout most compression regimes. In addition, CSSP-based methods exhibit significantly smoother degradation behavior as the FLOPs budget decreases, indicating better pruning stability.

An especially notable observation from Figure 8 is that when the remaining FLOPs ratio approaches approximately 40%, the performance of ℓ_1/ℓ_2 filter pruning begins to enter a plateau region. Beyond this point, further increases in retained FLOPs provide only limited performance recovery, while CSSP-based methods continue to maintain substantially better accuracy and lower test loss. Moreover, at the 40% FLOPs retention level, the performance of ℓ_1/ℓ_2 filter pruning has already degraded considerably, whereas the CSSP-based methods continue to maintain substantially better accuracy and lower test loss. This performance disparity makes the 40% FLOPs retention level an informative point for analyzing the structural differences between the pruning methods.

To further investigate these differences, Figure 9 illustrates the layer-wise FLOPs retention ratios when the

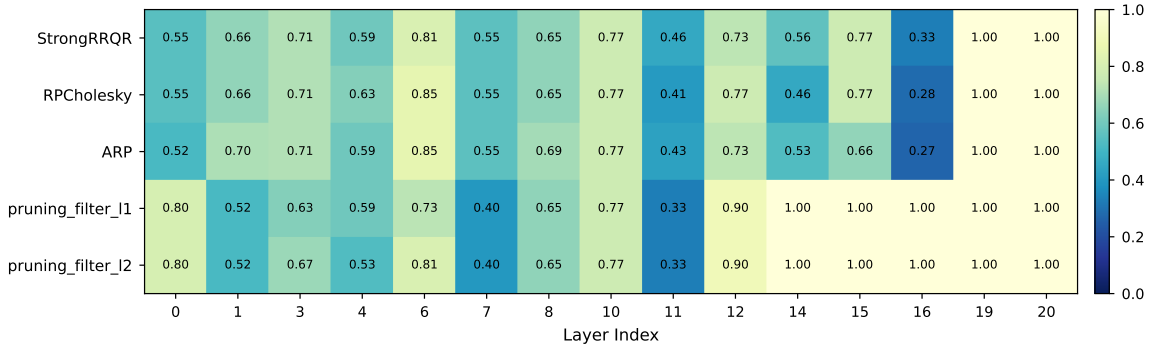

Figure 9: Layer-wise FLOPs retention ratios across different compression methods with 40% overall FLOPs retained. Each entry represents the proportion of FLOPs preserved in the corresponding layer after compression.

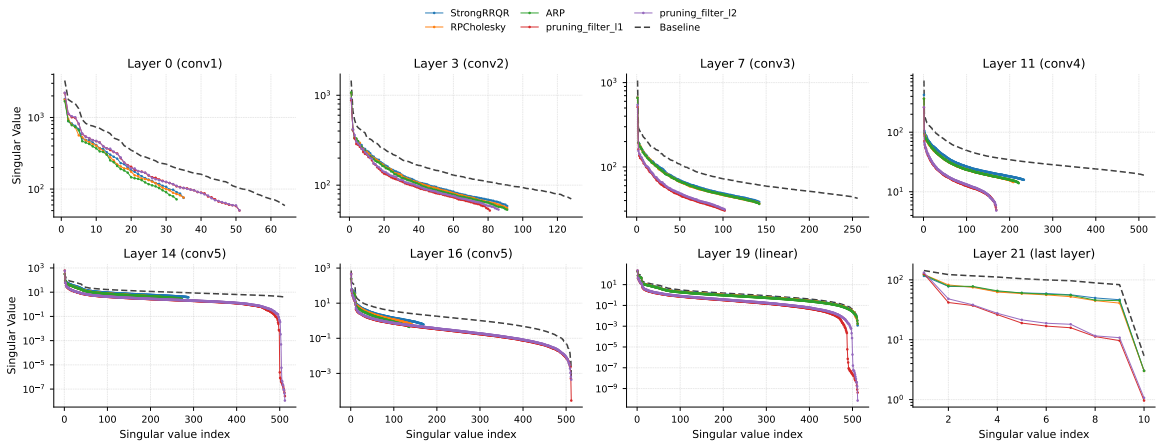

Figure 10: Singular value distributions of activation matrices across different layers under 40% FLOPs retention.

overall FLOPs budget is fixed at 40% of the original model. Meanwhile, Figure 10 shows the singular value spectra of the activation matrices in representative layers after pruning. From these results, we observe that CSSP-based methods are generally more effective at preserving the original rank structure of the activation matrices.

More specifically, in the early convolutional blocks (e.g., conv1 and conv2), both CSSP-based methods and ℓ_1/ℓ_2 filter pruning preserve relatively similar singular value distributions, suggesting that low-level feature extraction capabilities remain largely intact after pruning. However, noticeable differences begin to emerge in the intermediate convolutional blocks, where ℓ_1/ℓ_2 filter pruning starts to exhibit accelerated singular value decay. The discrepancy becomes substantially more pronounced in the deeper convolutional blocks, particularly in conv5. For example, in Layer 14, ℓ_1/ℓ_2 filter pruning produces significantly smaller trailing singular values compared with CSSP-based methods, implying severe degradation of the original activation subspace structure. Similar phenomena are also observed in the linear layers, where CSSP-based methods continue to maintain singular value distributions much closer to the uncompressed baseline model.

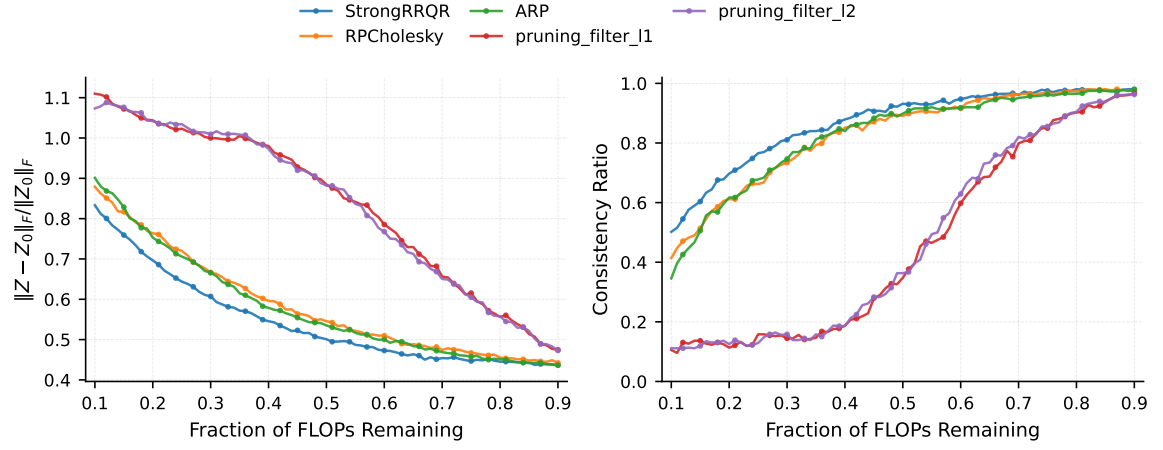

Figure 11: Relative Frobenius norm error $\|Z - Z_0\|_F / \|Z_0\|_F$ and consistency ratio of the last-layer activation matrix under different FLOPs retention ratios, where Z and Z_0 denote the last-layer activation matrices of the pruned and original models, respectively. Both Z and Z_0 have shape (batch size, 10), where 10 is the number of output classes. The consistency ratio measures the proportion of rows whose argmax indices are preserved between Z and Z_0.

As a result, consistent with the error propagation analysis in Section 4.2, CSSP-based methods tend to achieve smaller relative Frobenius errors with respect to the original last activation matrix (as shown in Figure 11), which further explains their superior accuracy, since the final predictions are determined by the row-wise argmax of the last activation matrix. Moreover, although the three CSSP-based methods exhibit

broadly similar behavior across all compression levels, StrongRRQR consistently achieves slightly lower relative Frobenius errors and higher consistency ratios than RPCholesky and ARP. This advantage is also reflected in its marginally better classification accuracy and test loss as shown in Figure 8.

5.2 CSSP-Based Pruning vs. Magnitude Pruning under Parameter Constraints

In this section, we compare three CSSP-based pruning methods with magnitude pruning under parameter constraints. As before, magnitude pruning is implemented within the same iterative pruning framework as Algorithm 6. The only difference lies in the compressibility term of the prunability score, which is computed from the relative magnitude of the weights selected for removal.

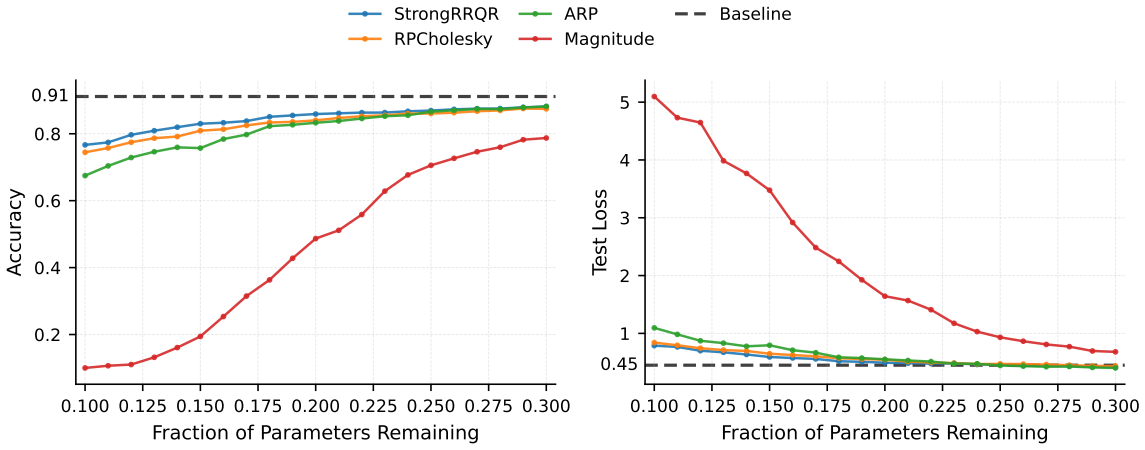

Figure 12: Performance comparison of different compression methods across varying parameters ratios. The left panel shows test accuracy, while the right panel shows test loss. The dashed line indicates the uncompressed baseline.

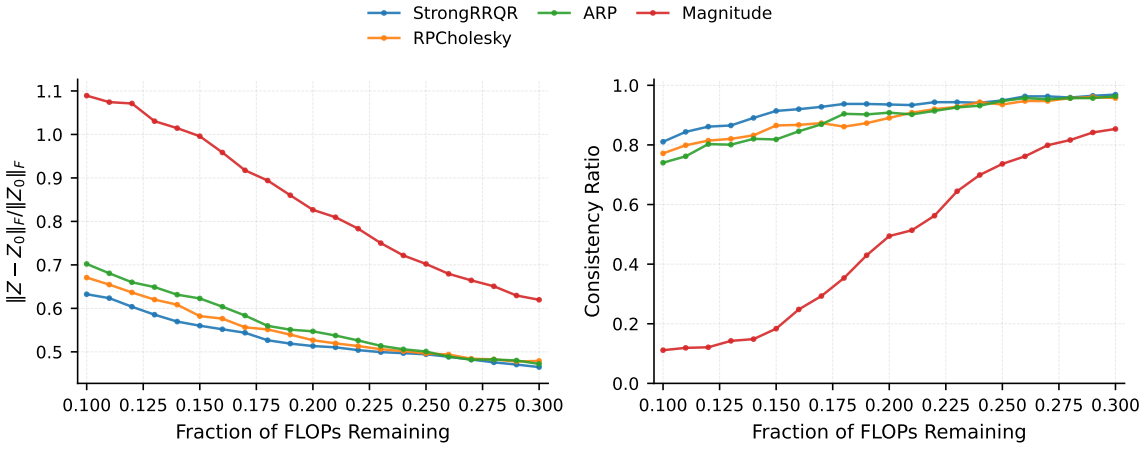

Figure 13: Relative Frobenius norm error $\|Z - Z_0\|_F / \|Z_0\|_F$ and consistency ratio of the last-layer activation matrix under different parameter retention ratios, where Z and Z_0 denote the last-layer activation matrices of the pruned and original models, respectively. Both Z and Z_0 have shape (batch size, 10), where 10 is the number of output classes. The consistency ratio measures the proportion of rows whose argmax indices are preserved between Z and Z_0.

The performance of the pruned models across varying parameter retention ratios is shown in Figure 12. Consistent with the relative Frobenius errors shown in Figure 13, CSSP-based methods achieve significantly better predictive performance than magnitude pruning. Combined with Figure 8 and Figure 11, we find that the CSSP-based pruning simultaneously outperforms structured methods on FLOPs efficiency and unstructured methods on parameter efficiency.

A notable observation is the substantial performance gap between CSSP-based methods and magnitude pruning under aggressive parameter compression. When only 10% – 15% of the parameters are retained, magnitude pruning suffers a dramatic degradation in both accuracy and test loss, whereas the CSSP-based methods continue to preserve a large fraction of the original model performance. This indicates that parameter importance measured solely by weight magnitude may fail to preserve the representational structure of the network under severe compression.

Moreover, both Figure 8 and Figure 12 suggest that the three CSSP-based methods exhibit highly similar performance across the entire range of parameter retention ratios, suggesting that the effectiveness of the framework is largely robust to the specific column-selection strategy employed.

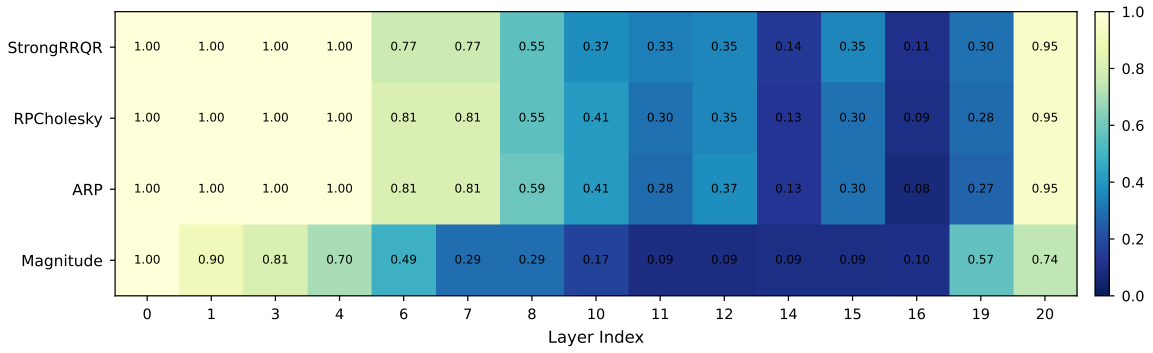

Figure 14: Layer-wise parameters retention ratios across different compression methods with 15% overall parameters retained. Each entry represents the proportion of parameters preserved in the corresponding layer after compression.

Figure 14 exhibits a pruning pattern different from that observed in Figure 9. Under parameter constraints, all methods tend to prune the conv4 and conv5 blocks (corresponding to Layers 10–16). This observation is consistent with the layer-wise parameter distribution summarized in Table 2, since these layers contain the majority of the model parameters.

Figure 15 shows the singular value distributions of activation matrices across representative layers under 15% parameter retention. The results indicate that although the pruning order differs between the two resource-constrained settings, the resulting singular value spectrum deviation patterns remain highly similar. In the early and intermediate layers, the singular value spectra remain relatively well aligned across different methods. However, noticeable deviations emerge in the last convolutional block (Layer 14) and the linear layers, following a similar spectrum deviation pattern to that observed in Figure 10.

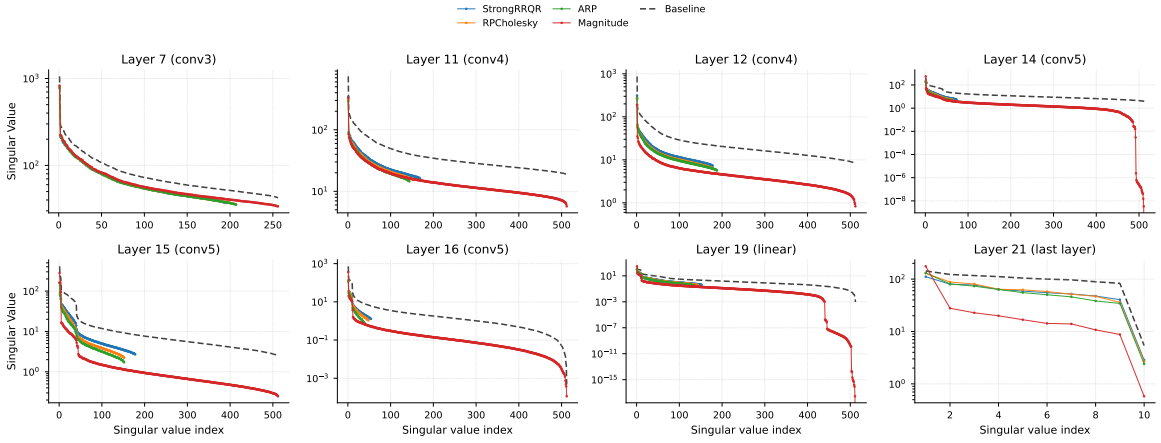

Figure 15: Singular value distributions of activation matrices across different layers under 15% parameters retention.

5.3 Rank Degradation in Filter Pruning and Magnitude Pruning

Another interesting phenomenon observed in our experiments is that rank degradation in both filter pruning and magnitude pruning follows a systematic propagation pattern that appears to be independent of the pruning order of the layers.

As shown in Figure 8 and Figure 12, the performance of ℓ_1/ℓ_2 filter pruning and magnitude pruning deteriorates rapidly when the FLOPs retention ratio decreases from approximately 70% to 40%, and when the parameter retention ratio decreases from approximately 30% to 15%, respectively. To better understand this phenomenon, Section A presents the relative singular value spectra (σ_i/σ_1) of activation matrices from representative layers under the aforementioned resource retention ratio range.

A notable observation is that the singular value deviations introduced by both ℓ_1/ℓ_2 filter pruning and magnitude pruning first become apparent in Layer 14 within the last convolutional block (conv5). As pruning becomes more aggressive, the degradation gradually propagates to Layer 15 and Layer 19 (the first linear layer), and subsequently to Layer 20, even though these layers are **not pruned** for ℓ_1/ℓ_2 filter pruning as shown in Figure 9. Eventually, the degradation further spreads to several earlier convolutional layers. Moreover, the degradation is not simply reflected by a reduction in the leading singular values, but more importantly by a substantial collapse of the trailing singular spectrum. This indicates that the model partially retains its primary feature directions but loses a considerable amount of lower-energy yet semantically meaningful representation subspaces.

The results also reveal that different layers exhibit different sensitivities to filter pruning and magnitude pruning. Earlier convolutional layers, such as conv1 and conv2, remain relatively stable even under aggressive compression, whereas deeper convolutional and linear layers are more vulnerable to rank collapse. This observation is consistent with the widely accepted interpretation of deep neural networks: shallow layers mainly capture low-level visual patterns such as edges and textures, while deeper layers encode progressively more abstract semantic representations.

Consequently, once the deeper convolutional blocks suffer severe rank degradation, the semantic representation capability of the network becomes substantially impaired. As a result, subsequent linear layers (Layers 19 and 20) can only operate on already degraded feature representations, leading to severe spectral distortion in the final activation space. Since the linear layers are responsible for integrating high-level semantic features and producing the final classification decisions, such representation degradation can substantially impair the predictive capability of the pruned model. This may explain the performance degradation observed in Figure 8 and Figure 12.

In contrast, CSSP-based methods preserve the relative singular value spectra much more faithfully across different pruning ratios. Even under aggressive resource constraints, their activation spectra remain closer to those of the uncompressed baseline model. This suggests that CSSP-based pruning better preserves the intrinsic low-rank structure and activation subspaces of the original network, which may explain their superior classification accuracy.

Understanding such representation dynamics during pruning provides useful insights into how different blocks in deep neural networks process, propagate, and reorganize information. By analyzing how semantic representations degrade and propagate across layers under compression, we can better understand the functional roles and sensitivities of different network components. More importantly, these observations may provide meaningful guidance for designing more efficient and parsimonious neural architectures, as well as developing pruning strategies that better preserve critical semantic representation structures and predictive capabilities.

6 Conclusion and Future Work

In this project, we investigated several CSSP-based pruning methods together with classical pruning approaches within a unified iterative pruning framework. We analyzed not only their final pruning performance, but also their finer-grained effects on the activation matrices throughout the network.

Our experiments suggest that preserving the final activation matrix is closely related to preserving the prediction performance of the original model. In particular, the Frobenius error between the original and pruned activation matrices provides a direct measure of representation preservation quality. This observation indicates that an effective pruning method should control the approximation error of intermediate activation matrices in order to limit error propagation across layers.

Moreover, through the pruning processes of ℓ_1/ℓ_2 filter pruning and magnitude pruning, we observed that different layers exhibit substantially different sensitivities to compression. In particular, preserving the singular value structure of certain deep convolutional and linear layers appears to be especially important for maintaining model accuracy. This observation suggests that hybrid pruning strategies may be more effective in practice. For example, in a VGG-16 architecture, computationally inexpensive methods such as ℓ_1/ℓ_2 filter pruning may be sufficient for early convolutional blocks, while more accurate CSSP-based methods can be applied to deeper convolutional blocks and linear layers in order to achieve both strong predictive performance and computational efficiency.

In this project, we focused primarily on fixed-rank column subset selection methods. However, many CSSP algorithms also admit adaptive-rank variants that automatically determine the approximation rank according to a prescribed approximation tolerance. Such approaches could allow the pruning process to become more fine-grained and controllable by directly constraining the Frobenius error of each layer's activation matrix.

Ideally, an adaptive pruning framework would automatically determine the maximum compression level that preserves the desired prediction performance while minimizing unnecessary redundancy in every layer. Investigating such adaptive-rank pruning strategies remains an important direction for future work.

Finally, practical deployment considerations should also be incorporated into pruning design. In real-world industrial settings, modern hardware accelerators such as GPUs often favor neuron or channel dimensions that are multiples of 8 or 16 for improved computational efficiency. Consequently, integrating hardware-aware structural constraints into CSSP-based pruning frameworks is another important direction for future research.

References

[1] J. Chee, M. Flynn (née Renz), A. Damle, and C. De Sa, *Model preserving compression for neural networks*, in Advances in Neural Information Processing Systems, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, eds., vol. 35, Curran Associates, Inc., 2022, pp. 38060–38074.

[2] Y. Chen, E. N. Epperly, J. A. Tropp, and R. J. Webber, *Randomly pivoted Cholesky: practical approximation of a kernel matrix with few entry evaluations*, Comm. Pure Appl. Math., 78 (2025), pp. 995–1041.

[3] A. Cortinovis and D. Kressner, *Adaptive randomized pivoting for column subset selection, DEIM, and low-rank approximation*, SIAM J. Matrix Anal. Appl., 47 (2026), pp. 25–47.

[4] A. Deshpande and L. Rademacher, *Efficient volume sampling for row/column subset selection*, in 2010 IEEE 51st Annual Symposium on Foundations of Computer Science—FOCS 2010, IEEE Computer Soc., Los Alamitos, CA, 2010, pp. 329–338.

[5] A. Deshpande, L. Rademacher, S. Vempala, and G. Wang, *Matrix approximation and projective clustering via volume sampling*, in Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms, ACM, New York, 2006, pp. 1117–1126.

[6] G. H. Golub and C. F. Van Loan, *Matrix computations*, Johns Hopkins Studies in the Mathematical Sciences, Johns Hopkins University Press, Baltimore, MD, third ed., 1996.

[7] M. Gu and S. C. Eisenstat, *Efficient algorithms for computing a strong rank-revealing QR factorization*, SIAM J. Sci. Comput., 17 (1996), pp. 848–869.

[8] S. Han, J. Pool, J. Tran, and W. J. Dally, *Learning both weights and connections for efficient neural networks*, in Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 1, NIPS'15, Cambridge, MA, USA, 2015, MIT Press, p. 1135–1143.

[9] G. Hinton, O. Vinyals, and J. Dean, *Distilling the knowledge in a neural network*, 2015.

[10] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. G. Howard, H. Adam, and D. Kalenichenko, *Quantization and training of neural networks for efficient integer-arithmetic-only inference*, 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, (2017), pp. 2704–2713.

[11] A. Krizhevsky, G. Hinton, et al., *Learning multiple layers of features from tiny images.(2009)*, 2009.

[12] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, *Pruning filters for efficient convnets*, in International Conference on Learning Representations, 2017.

[13] K. Simonyan and A. Zisserman, *Very deep convolutional networks for large-scale image recognition*, arXiv preprint arXiv:1409.1556, (2014).

Appendix A Rank Degradation During Pruning

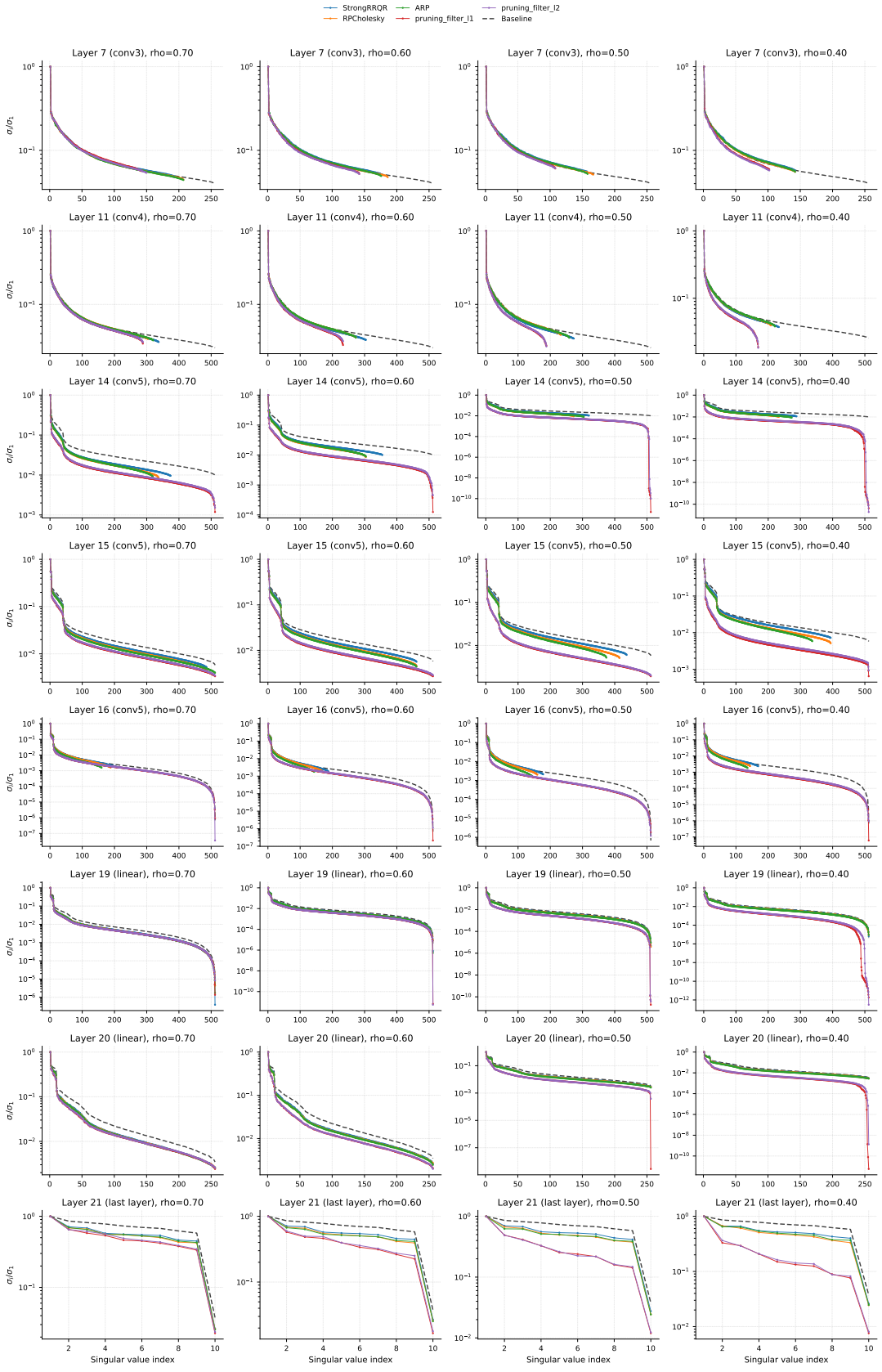

Figure 16: Evolution of activation singular value spectra across representative layers under varying FLOPs retention ratios.

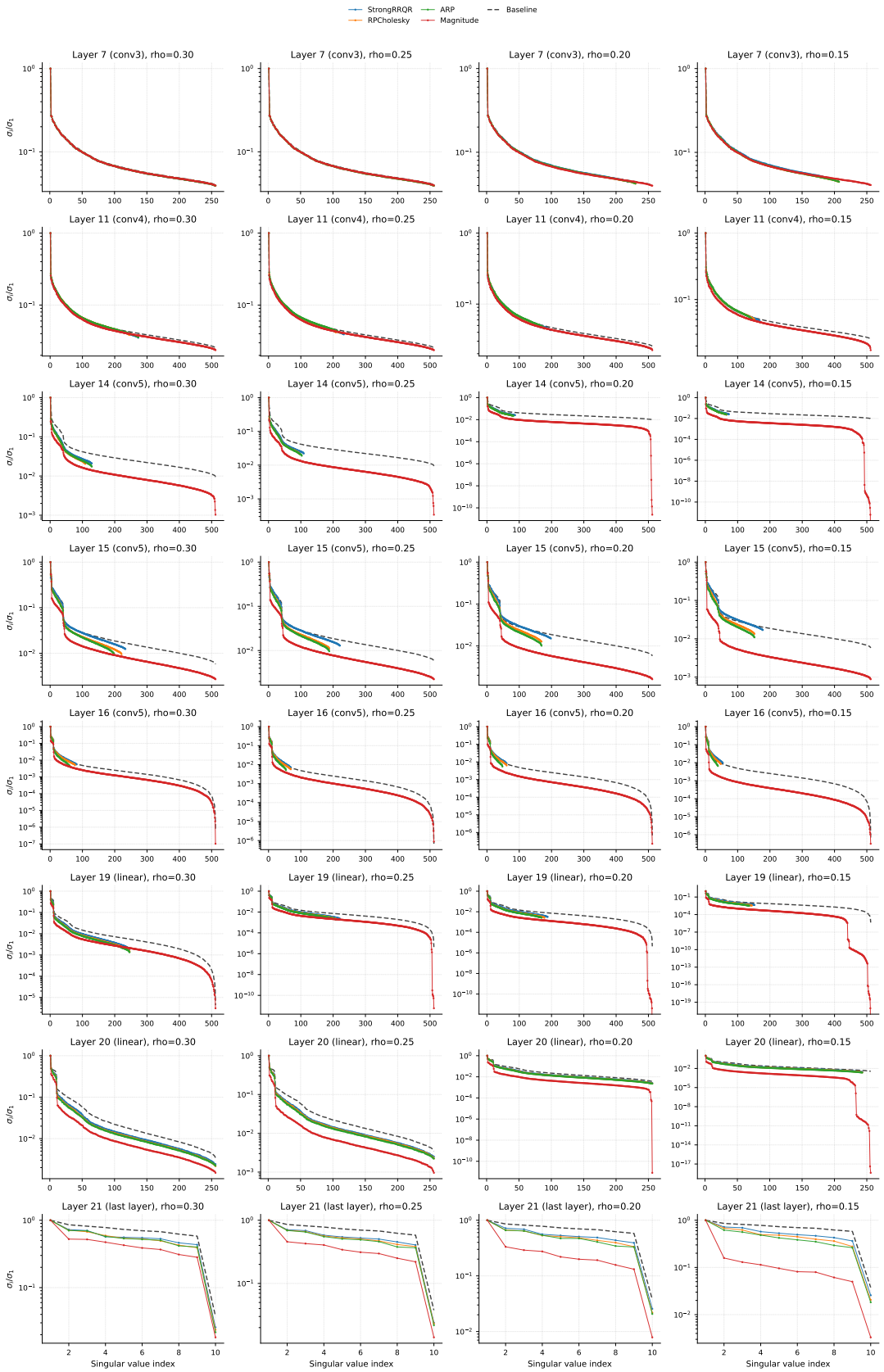

Figure 17: Evolution of activation singular value spectra across representative layers under varying parameter retention ratios.

Appendix B Usage of the large language model in the project

ChatGPT was used as an assistant tool during the preparation of this project.

Report.

- ChatGPT assisted in refining Section 1 and Section 2.3 by suggesting additional background material on neural network compression techniques.
- The LaTeX code used to generate Figure 7 was adapted from `https://github.com/HarisIqbal88/PlotNeuralNet` and modified with the assistance of ChatGPT.
- Table 2 was generated with the assistance of ChatGPT.
- The discussion of the effects of pruning on semantic representations in Section 5.3 was informed by discussions with ChatGPT.
- The discussion of GPU-related and practical deployment considerations for neural network pruning in Section 6 was informed by discussions with ChatGPT.

Code.

- ChatGPT assisted in refining the implementation of `Plot.py`.